

// Jivops

Nomad en Producción

Guía completa — 30 lecciones

Fundamentos · Jobs y scheduling · Recursos y escalado · Networking ·
Multi-región y multi-cloud · Producción avanzada

Contenido

Módulo 1: Fundamentos de Nomad

- Lección 01 — Qué es Nomad y en qué se diferencia de Kubernetes
- Lección 02 — Arquitectura de Nomad — servers y clients
- Lección 03 — Instalación de un cluster de Nomad
- Lección 04 — Job specification — el HCL de Nomad
- Lección 05 — Nomad UI y CLI — herramientas de operación

Módulo 2: Jobs y scheduling

- Lección 06 — Tipos de jobs — service, batch, system, sysbatch
- Lección 07 — Task groups y tasks
- Lección 08 — Drivers de ejecución — Docker, exec, java, raw_exec
- Lección 09 — El scheduler de Nomad — bin packing y constraints
- Lección 10 — Actualizaciones de jobs — rolling updates y canary

Módulo 3: Recursos y escalado

- Lección 11 — Asignación de recursos — CPU, memoria, red
- Lección 12 — Autoscaling con el Nomad Autoscaler
- Lección 13 — Affinity y anti-affinity
- Lección 14 — Spread — distribuir tareas entre datacenters
- Lección 15 — Node pools y gestión de capacidad

Módulo 4: Networking y descubrimiento de servicios

- Lección 16 — Service discovery nativo de Nomad
- Lección 17 — Integración con Consul para service mesh
- Lección 18 — Nomad Connect — sidecar proxies
- Lección 19 — Port mapping y networking de tareas
- Lección 20 — Integración con balanceadores de carga externos

Módulo 5: Multi-región y multi-cloud

- Lección 21 — Federación multi-región en Nomad
- Lección 22 — Despliegues multi-cloud con Nomad
- Lección 23 — Namespaces en Nomad
- Lección 24 — Nomad vs Kubernetes — cuándo elegir cada uno
- Lección 25 — Migración de cargas desde Kubernetes hacia Nomad

Módulo 6: Producción avanzada

Lección 26 — Seguridad — ACLs en Nomad

Lección 27 — Integración con Vault para gestión de secretos

Lección 28 — Monitoreo de Nomad con Prometheus

Lección 29 — Backup y disaster recovery de Nomad

Lección 30 — Proyecto final: plataforma Nomad completa lista para producción

Módulo 1: Fundamentos de Nomad

Lección 01 — Qué es Nomad y en qué se diferencia de Kubernetes

Nomad se enfoca específicamente en la orquestación de cargas de trabajo (scheduling y gestión del ciclo de vida de tareas), con un binario único y una superficie más simple, dejando capacidades como networking avanzado o gestión de secretos a herramientas complementarias como Consul y Vault, en vez de incluir todo eso dentro del núcleo.

Si el equipo solo necesita programar y ejecutar tareas de forma confiable sin requerir las capacidades más avanzadas de networking, extensibilidad o el ecosistema completo que Kubernetes ofrece, la superficie más simple y reducida de Nomad significa menos conceptos que aprender y menos complejidad operativa que mantener, evitando la sobrecarga de un sistema más amplio diseñado para cubrir casos de uso que ese equipo específico no necesita.

Lección 02 — Arquitectura de Nomad — servers y clients

Los servers forman el plano de control, tomando las decisiones de scheduling y manteniendo el estado del cluster mediante consenso, mientras los clients son los nodos de trabajo donde realmente se ejecutan las tareas asignadas por los servers.

Añadir más nodos client aumenta directamente la capacidad de cómputo disponible para ejecutar tareas, sin necesitar tocar ni complicar el número relativamente reducido y estable de nodos server que mantienen el consenso del plano de control; esa separación de responsabilidades permite escalar la capacidad de ejecución del cluster de forma independiente, sin comprometer la estabilidad del mecanismo de coordinación central.

Lección 03 — Instalación de un cluster de Nomad

Nomad se despliega copiando y ejecutando un único binario tanto en los nodos server como en los nodos client, con la configuración determinando el rol que cumple cada instancia, simplificando significativamente el proceso de instalación comparado con coordinar múltiples componentes separados.

Con un único binario para distribuir y actualizar, no hay que preocuparse por verificar que múltiples componentes separados y distintos sean compatibles entre sí en cada actualización; esa simplicidad reduce directamente la superficie de error durante el proceso de instalación o actualización, comparado con coordinar cuidadosamente varios componentes separados que deben mantenerse sincronizados en versiones compatibles entre ellos.

Lección 04 — Job specification — el HCL de Nomad

HCL (HashiCorp Configuration Language), un lenguaje de configuración declarativo diseñado específicamente para ser legible tanto por humanos como procesable por máquinas, usado de forma consistente en otras herramientas del ecosistema HashiCorp como Terraform.

Un equipo que ya sabe leer y escribir HCL gracias a su experiencia previa con Terraform puede aplicar directamente ese mismo conocimiento de sintaxis general al aprender Nomad, sin tener que aprender un lenguaje de configuración completamente nuevo y distinto desde cero; esa consistencia entre herramientas del mismo ecosistema reduce significativamente la curva de aprendizaje al adoptar una herramienta relacionada nueva.

Lección 05 — Nomad UI y CLI — herramientas de operación

La UI ofrece una vista visual del estado del cluster, los jobs en ejecución y sus logs, facilitando la exploración y el diagnóstico visual, mientras la CLI permite automatizar tareas repetitivas, integrarse con scripts, y ejecutar operaciones desde pipelines de CI/CD de forma programática.

Un pipeline de CI/CD automatizado necesita ejecutar la operación de despliegue sin ninguna intervención manual humana en cada ejecución del flujo; la interfaz web requiere que una persona física interactúe haciendo clic, algo incompatible con un proceso completamente automatizado, mientras la CLI puede invocarse directamente desde un script como cualquier otro comando programático, sin necesitar esa interacción humana en cada despliegue.

Módulo 2: Jobs y scheduling

Lección 06 — Tipos de jobs — service, batch, system, sysbatch

Un job service está pensado para ejecutarse de forma continua y de larga duración (como un servidor web que siempre debe estar activo), mientras un job batch está pensado para completar una tarea específica y finita, terminando su ejecución una vez que el trabajo se completa.

Generar un reporte mensual es una tarea con un inicio y un final claramente definidos (procesar los datos, generar el archivo, terminar), sin necesitar permanecer activa indefinidamente como lo haría un servidor web; esa naturaleza finita coincide directamente con el comportamiento esperado de un job batch, mientras un job service está pensado específicamente para procesos continuos que deben seguir ejecutándose de forma persistente sin un punto de finalización natural.

Lección 07 — Task groups y tasks

Un task group agrupa una o más tasks relacionadas que deben programarse juntas en el mismo nodo client (compartiendo recursos de red si es necesario), mientras cada task individual representa una unidad específica de trabajo (como ejecutar un contenedor concreto) dentro de ese grupo.

El task group garantiza que todas sus tasks se programen siempre juntas en el mismo nodo client, sin excepción; si la task de la aplicación principal y la task del sidecar estuvieran en task groups separados, el scheduler de Nomad podría programarlas en nodos client completamente distintos, rompiendo la comunicación directa y local que ese patrón de sidecar necesita para funcionar correctamente entre ambas tasks relacionadas.

Lección 08 — Drivers de ejecución — Docker, exec, java, raw_exec

Define el mecanismo real mediante el cual Nomad ejecuta esa tarea específica (como lanzar un contenedor Docker, ejecutar un binario aislado en un namespace de Linux, o ejecutar un proceso de Java), permitiendo que Nomad soporte distintos tipos de cargas de trabajo más allá de únicamente contenedores.

Si Nomad solo soportara el driver Docker, cualquier aplicación que no estuviera ya empaquetada como contenedor tendría que containerizarse primero antes de poder orquestarse; al soportar también drivers como exec (para binarios tradicionales ejecutados directamente), Nomad puede gestionar una variedad más amplia de cargas de trabajo distintas sin forzar esa conversión previa a contenedores en todos los casos.

Lección 09 — El scheduler de Nomad — bin packing y constraints

Por defecto, Nomad usa una estrategia de bin packing que intenta concentrar las tareas en el menor número de nodos posible para maximizar la utilización de recursos, mientras un constraint permite restringir explícitamente en qué nodos específicos (según atributos como el sistema operativo o una etiqueta personalizada) puede programarse una tarea determinada.

Sin esa restricción explícita, el scheduler de Nomad podría considerar un nodo con un sistema operativo incompatible como candidato válido para esa tarea, resultando en un fallo real al intentar ejecutarla ahí; el constraint filtra de antemano los nodos candidatos, garantizando que solo los nodos realmente compatibles con los requisitos específicos de esa tarea sean considerados como destino válido para la colocación.

Lección 10 — Actualizaciones de jobs — rolling updates y canary

Reemplaza las instancias del job de forma gradual y controlada (actualizando un subconjunto a la vez según un parámetro de 'max_parallel'), manteniendo cierta capacidad del servicio disponible durante todo el proceso, en vez de causar una interrupción completa al reemplazar todas las instancias simultáneamente.

Si todas las instancias se detuvieran y reemplazaran al mismo tiempo, habría un período real donde ninguna instancia del servicio estaría disponible para atender peticiones, causando una interrupción completa visible para los usuarios durante ese lapso; una estrategia de rolling update mantiene siempre algunas instancias de la versión anterior activas mientras se actualizan las demás gradualmente, evitando ese período de indisponibilidad total.

Módulo 3: Recursos y escalado

Lección 11 — Asignación de recursos — CPU, memoria, red

Le indica al scheduler cuánta capacidad reservar para esa task específica al decidir en qué nodo colocarla, evitando sobrecargar un nodo client con más trabajo del que realmente puede soportar según su capacidad real disponible.

Sin conocer cuánta memoria requiere realmente cada task, el scheduler no tendría forma de saber si un nodo client específico tiene capacidad suficiente disponible antes de colocar esa tarea ahí; colocar tareas sin considerar esa información podría sobrecargar un nodo más allá de su capacidad física real, causando problemas de rendimiento o fallos en las tareas que terminan compitiendo por recursos insuficientes en ese mismo nodo sobrecargado.

Lección 12 — Autoscaling con el Nomad Autoscaler

Ajusta automáticamente el número de instancias del job hacia arriba o hacia abajo, según el valor observado de la métrica configurada comparado con los umbrales definidos en la política, sin requerir que un humano intervenga manualmente cada vez que cambia la demanda real.

Si un operador humano solo revisara las métricas cada hora, existiría una ventana de tiempo real donde la demanda pudo haber cambiado significativamente sin que nadie ajustara la capacidad a tiempo, además de que ninguna persona puede estar monitoreando constantemente las 24 horas del día; el autoscaling automático responde de forma continua y mucho más rápida a los cambios reales de demanda, sin depender de la disponibilidad humana ni de intervalos fijos de revisión manual.

Lección 13 — Affinity y anti-affinity

Una regla de affinity expresa una preferencia por colocar ciertas tareas cerca entre sí (como en el mismo datacenter), mientras una regla de anti-affinity expresa la preferencia opuesta, de mantener ciertas tareas distribuidas en nodos o ubicaciones distintas (útil para evitar que todas las réplicas de un servicio caigan juntas si falla un único nodo).

Si todas las réplicas de un servicio crítico terminaran coincidentalmente en el mismo nodo (sin ninguna regla que lo evite), la falla de ese único nodo eliminaría absolutamente todas las réplicas del servicio a la vez, causando una interrupción total; la regla de anti-affinity guía deliberadamente al scheduler hacia una distribución más resiliente entre nodos distintos, garantizando que la falla de uno solo no afecte simultáneamente a todas las réplicas del mismo servicio.

Lección 14 — Spread — distribuir tareas entre datacenters

Spread permite especificar explícitamente una distribución proporcional deseada entre distintos valores de un atributo (como qué porcentaje de instancias debe ir a cada datacenter específico), ofreciendo un control más preciso sobre la proporción exacta de la distribución, más allá de solo preferir evitar la concentración.

Con spread se puede especificar una proporción exacta y deliberada (como 60/40 entre dos datacenters específicos), útil si se necesita ese control preciso por razones de capacidad o costo relativo entre ubicaciones; anti-affinity solo expresa la preferencia general de evitar que las instancias se concentren juntas, sin ofrecer ese mismo nivel de control sobre una proporción exacta y específica entre las distintas ubicaciones disponibles.

Lección 15 — Node pools y gestión de capacidad

Permite dirigir cargas de trabajo específicas (como tareas que requieren GPU) hacia el subconjunto de nodos que realmente tienen esa capacidad especializada, evitando que el scheduler considere nodos sin esa capacidad como candidatos válidos para esas tareas específicas.

Sin esa agrupación explícita por node pool, el scheduler de Nomad podría considerar erróneamente un nodo estándar sin GPU como candidato válido para una tarea que específicamente requiere esa capacidad, resultando en un fallo real al intentar ejecutar esa tarea en un nodo que no tiene el hardware necesario; organizar los nodos en pools separados dirige la tarea únicamente hacia el subconjunto de nodos que genuinamente tienen la capacidad especializada requerida.

Módulo 4: Networking y descubrimiento de servicios

Lección 16 — Service discovery nativo de Nomad

Permite que otras tareas o servicios del cluster encuentren dinámicamente la ubicación real (dirección IP y puerto) de un servicio específico sin necesitar direcciones fijas hardcodeadas, algo especialmente importante dado que las tareas pueden reprogramarse hacia nodos distintos con direcciones distintas en cualquier momento.

La dirección IP hardcodeada apuntaría específicamente a la ubicación original donde el servicio B estaba corriendo en ese momento; si Nomad reprograma ese servicio hacia un nodo client completamente distinto tras una falla, esa dirección original quedaría desactualizada y el servicio A seguiría intentando conectarse a un lugar donde el servicio B ya no está, mientras el service discovery dinámico resolvería ese problema consultando siempre la ubicación real y actual en vez de depender de una dirección fija que puede volverse obsoleta.

Lección 17 — Integración con Consul para service mesh

Consul aporta capacidades más avanzadas como health checking más sofisticado, un almacén de configuración distribuido (key-value store), y la base para construir un service mesh completo con cifrado automático y políticas de tráfico entre servicios, extendiendo lo que el service discovery básico de Nomad ofrece por sí mismo.

El service discovery básico de Nomad resuelve el problema de encontrar la ubicación de un servicio, pero no incluye por sí solo capacidades de cifrado automático del tráfico entre esos servicios; si la organización tiene un requisito estricto de que todo el tráfico interno esté cifrado, necesitaría las capacidades adicionales de service mesh que Consul aporta específicamente, extendiendo lo que el descubrimiento básico de Nomad no cubre por sí mismo.

Lección 18 — Nomad Connect — sidecar proxies

Inyecta automáticamente un proxy sidecar junto a la task principal dentro del mismo task group, que gestiona de forma transparente el tráfico cifrado entrante y saliente de esa tarea hacia otros servicios del mesh, sin que la aplicación misma necesite implementar directamente esa lógica de cifrado o descubrimiento.

El desarrollador de cada aplicación puede enfocarse exclusivamente en la lógica de negocio real, sin tener que implementar manualmente y de forma correcta la lógica de cifrado de red dentro de su propio código; delegar esa responsabilidad hacia el proxy sidecar (una capa de infraestructura compartida y ya probada) simplifica significativamente el desarrollo, además de garantizar consistencia en cómo se aplica el cifrado en todo el mesh, sin depender de que cada equipo lo implemente correctamente por separado.

Lección 19 — Port mapping y networking de tareas

Define qué puerto interno usa la aplicación dentro de la task y cómo se expone (o traduce) hacia un puerto accesible desde fuera de esa task, permitiendo que Nomad coordine correctamente las conexiones de red hacia esa aplicación específica sin conflictos con otras tareas que podrían usar el mismo puerto interno.

El sistema operativo de un nodo no permite que dos procesos distintos escuchen simultáneamente en exactamente el mismo puerto de red sin generar un conflicto real; si Nomad no coordinara adecuadamente el mapeo de puertos entre tareas que terminan en el mismo nodo client, una de esas dos tareas fallaría al intentar iniciar, ya que el puerto que necesita ya estaría ocupado por la otra tarea que llegó primero a ese

mismo nodo.

Lección 20 — Integración con balanceadores de carga externos

El balanceador necesita una forma de mantenerse actualizado sobre dónde están realmente ejecutándose las instancias del servicio en cada momento, ya que una configuración estática de destinos fijos quedaría desactualizada tan pronto como Nomad re programe alguna tarea hacia un nodo distinto.

El service discovery de Nomad conoce en tiempo real dónde está realmente ejecutándose cada instancia del servicio en cada momento específico; conectar el balanceador externo directamente a esa fuente dinámica de información le permite recibir actualizaciones automáticas cuando una tarea se mueve hacia otro nodo, algo que una lista fija de destinos configurada manualmente nunca podría lograr sin requerir intervención manual constante cada vez que ocurre ese tipo de cambio real.

Módulo 5: Multi-región y multi-cloud

Lección 21 — Federación multi-región en Nomad

Permite gestionar y visualizar múltiples clusters regionales distintos desde un único punto de control, y desplegar jobs específicamente dirigidos hacia una región particular, sin necesitar mantener herramientas de administración completamente separadas para cada región individual.

Administrar cada cluster regional de forma completamente aislada requeriría mantener herramientas de gestión, credenciales y procesos operativos separados y duplicados para cada región; federar ambos clusters centraliza la visibilidad y el control, permitiendo gestionar y desplegar jobs hacia cualquiera de las regiones desde un único punto de control, reduciendo significativamente esa complejidad operativa de coordinar múltiples clusters regionales por separado.

Lección 22 — Despliegues multi-cloud con Nomad

Al ser una herramienta de orquestación agnóstica de la infraestructura subyacente, Nomad puede desplegarse de forma consistente sobre máquinas de cualquier proveedor cloud o incluso infraestructura on-premise, permitiendo usar el mismo modelo de operación y las mismas especificaciones de job independientemente de dónde viva físicamente cada nodo.

Si la misma especificación de job y el mismo modelo de operación de Nomad funcionan consistentemente tanto en AWS como en el datacenter on-premise propio, el equipo no necesita duplicar su conocimiento y procesos operativos aprendiendo herramientas de orquestación completamente distintas para cada entorno; esa consistencia reduce significativamente la complejidad real de gestionar cargas de trabajo distribuidas entre múltiples ubicaciones y proveedores de infraestructura distintos.

Lección 23 — Namespaces en Nomad

Proveen un aislamiento lógico entre distintos grupos de jobs, permitiendo que jobs con el mismo nombre existan simultáneamente en namespaces distintos sin conflicto, y facilitando aplicar políticas de acceso (ACLs) diferenciadas según el namespace específico.

Aunque ambos entornos compartan la misma infraestructura física del cluster, el aislamiento lógico de los namespaces mantiene separados los jobs de desarrollo de los de producción, y las políticas de acceso diferenciadas pueden restringir quién tiene permiso de modificar jobs específicamente dentro del namespace de producción; un cambio accidental realizado dentro del namespace de desarrollo queda contenido ahí, sin ninguna forma directa de afectar jobs que viven en el namespace separado de producción.

Lección 24 — Nomad vs Kubernetes — cuándo elegir cada uno

Evaluar si el equipo necesita las capacidades avanzadas y el ecosistema extenso que Kubernetes ofrece (a cambio de mayor complejidad operativa), o si prefiere la simplicidad operativa de Nomad combinada selectivamente con herramientas complementarias como Consul y Vault solo donde realmente se necesiten.

Un equipo pequeño sin experiencia previa en orquestación, cuya necesidad real es desplegar algunas aplicaciones de forma confiable sin requerir el ecosistema completo y la complejidad operativa adicional de Kubernetes, podría beneficiarse de la curva de aprendizaje más suave y la superficie más simple de Nomad, evitando la sobrecarga de aprender capacidades avanzadas que ese proyecto específico no necesita usar todavía en su etapa actual.

Lección 25 — Migración de cargas desde Kubernetes hacia Nomad

Traducir los conceptos equivalentes entre ambas plataformas (como un Deployment de Kubernetes hacia un job de tipo service en Nomad, o un ConfigMap hacia la gestión de configuración de Nomad), validando que el comportamiento resultante sea realmente equivalente antes de considerar la migración completa.

Aunque un Deployment de Kubernetes y un job de tipo service de Nomad cumplen un propósito general similar, cada plataforma puede manejar detalles específicos de implementación (como reinicios, actualizaciones graduales, o health checks) de forma ligeramente distinta; asumir una traducción automática sin validar cuidadosamente el comportamiento resultante podría introducir diferencias sutiles no anticipadas, que solo se descubrirían como un problema real una vez que la aplicación ya esté migrada y corriendo en producción bajo Nomad.

Módulo 6: Producción avanzada

Lección 26 — Seguridad — ACLs en Nomad

Define qué usuarios o tokens tienen permiso para realizar qué operaciones específicas (como desplegar un job, leer logs, o administrar namespaces), aplicando el principio de menor privilegio para evitar que cualquiera con acceso al cluster pueda realizar cualquier operación sin ninguna restricción real.

Sin ACLs habilitadas, cualquier persona con conectividad de red hacia el cluster podría potencialmente ejecutar operaciones críticas (desplegar, modificar, eliminar jobs) sin ninguna verificación de si realmente tiene autorización para hacerlo; eso representaría un riesgo real tanto de errores accidentales como de acciones maliciosas deliberadas afectando directamente cargas de trabajo de producción, siendo precisamente el escenario que las ACLs están diseñadas para prevenir.

Lección 27 — Integración con Vault para gestión de secretos

Permite que Nomad inyecte secretos directamente desde Vault hacia la task en tiempo de ejecución, evitando tener que hardcodear esos valores sensibles directamente en la especificación del job (que podría quedar versionada en un repositorio de control de versiones de forma insegura).

Una vez que ese archivo con la contraseña hardcodeada se sube a un repositorio de control de versiones como Git, queda registrado permanentemente en el historial completo del repositorio; incluso si alguien eliminara esa línea en un commit posterior, cualquier persona con acceso al historial completo del repositorio podría seguir viendo esa contraseña expuesta, un riesgo real que la integración con Vault evita al nunca almacenar ese valor sensible directamente dentro del archivo versionado.

Lección 28 — Monitoreo de Nomad con Prometheus

Métricas sobre el estado de los jobs (cuántos están corriendo correctamente vs fallando), la utilización de recursos de los nodos client, el estado del consenso entre los servers, y la latencia de las decisiones de scheduling, dando visibilidad real del comportamiento operativo del cluster completo.

El servicio de Nomad podría estar técnicamente en línea y respondiendo normalmente, mientras varios jobs específicos fallan repetidamente por otras causas (como un error de configuración o un recurso insuficiente); esa disponibilidad técnica básica del propio Nomad es independiente de si las cargas de trabajo específicas que gestiona están funcionando correctamente, razón por la cual monitorear la métrica real de jobs fallando revela un problema operativo que la simple verificación de disponibilidad del servicio no detectaría.

Lección 29 — Backup y disaster recovery de Nomad

Si el estado del cluster se corrompiera o se perdiera por completo (por ejemplo, una falla grave de almacenamiento en los servers), sin un backup disponible habría que reconstruir manualmente todas las especificaciones de jobs y la configuración desde cero, un proceso lento que extendería significativamente el tiempo de recuperación ante un desastre real.

Reconstruir manualmente desde cero requeriría recordar y volver a escribir exactamente cada especificación de job tal como estaba configurada originalmente, un proceso lento y propenso a errores humanos; restaurar desde un backup ya existente permite recuperar el estado exacto del cluster en mucho menos tiempo, sin depender de la memoria humana para recrear cada detalle de configuración, una diferencia de tiempo real significativa durante una situación de desastre donde la rapidez de recuperación realmente importa.

Lección 30 — Proyecto final: plataforma Nomad completa lista para producción

Cluster con servers en alta disponibilidad + jobs bien especificados con estrategias de actualización apropiadas + service discovery con Consul + secretos gestionados con Vault + ACLs habilitadas + monitoreo con Prometheus y backup robusto del estado del cluster.

Intenta mencionar al menos dos pilares: alta disponibilidad de servers, service discovery con Consul, gestión de secretos con Vault o ACLs con monitoreo y backup.